Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа № 2

По дисциплине: «Криптография и сетевая безопасность»

Тема: «Избыточное помехоустойчивое кодирование данных: коды Хемминга и многомерные итеративные коды»

Выполнил:
Студент 4 курса
Группы АС-65
Романюк Д.А.
Проверил:
Хацкевич А.С.

Брест 2026

Цель работы

Исследовать теоретические основы и программную реализацию линейных систематических кодов Хемминга и многомерных итеративных кодов (кодов произведения), методы построения проверочных матриц и вычисления избыточных символов, алгоритмы локализации и исправления ошибок на основе анализа синдромов и взаимных паритетов, а также экспериментально оценить их корректирующую способность при различных кратностях ошибок.

Индивидуальное задание (Вариант №5)

Часть 1. Коды Хемминга:

1. На основе текстового файла либо строки сформировать бинарное информационное слово длиной не менее 16 бит (для согласованности принято k = 40 бит / 5 байт ASCII).
2. Построить проверочную матрицу Хемминга H = [P' | I_r] в каноническом виде по алгоритму возрастания веса столбцов (d_min = 3), а также исследовать расширенный код Хемминга с проверкой на четность (d_min = 4).
3. Вычислить избыточные символы X_r и кодовое слово X_n = [X_k | X_r].
4. Внести в кодовое слово случайные ошибки кратности 0, 1 и 2.
5. При декодировании принятого слова Y_n = [Y_k | Y_r] повторно вычислить избыточные символы Y'_r, найти синдром S = Y_r $\oplus$ Y'_r, локализовать и исправить одиночные ошибки (Y'_n = Y_n $\oplus$ E_n) и проанализировать поведение при 2 ошибках.

Часть 2. Многомерные итеративные коды (k = 40 бит):

В соответствии с Таблицей 5.2 методических указаний для Варианта №5 (K = 40 бит) исследовать 4 конфигурации матричной структуры:

1) 2D матрица: k1 = 5, k2 = 8, z = 1 (группы паритетов: 2; 3);
2) 2D матрица: k1 = 4, k2 = 10, z = 1 (группы паритетов: 2; 3);
3) 3D матрица: k1 = 5, k2 = 4, z = 2 (группы паритетов: 2; 3; 4; 5);
4) 3D матрица: k1 = 2, k2 = 10, z = 2 (группы паритетов: 2; 3; 4; 5).

Для каждой структуры сформировать паритеты по заданным группам (горизонтальные, вертикальные, диагональные, межплоскостные Z-проверки и контрольную сумму X_hv). Реализовать алгоритм обнаружения и исправления ошибок. Провести статистический анализ (N1 = 300 испытаний на кратность i = 1..5) с вычислением показателей N2/N1 (обнаружение/идентификация кратности) и N3/N1 (полная коррекция).

Краткие теоретические сведения

1. Коды Хемминга

Код Хемминга относится к классу линейных систематических блоковых кодов. Для защиты k информационных бит требуется r проверочных бит, удовлетворяющих неравенству 2^r >= k + r + 1. Проверочная матрица H размерности (r x n) формируется в каноническом виде H = [P' | I_r], где I_r — единичная матрица (r x r), а P' — подматрица (r x k), столбцы которой представляют собой двоичные векторы веса wt >= 2, упорядоченные по возрастанию веса.
Избыточные символы формируются умножением информационного вектора на транспонированную подматрицу: X_r = X_k * P'^T (mod 2). При приеме слова Y_n = [Y_k | Y_r] синдром вычисляется как S = Y_r ⊕ Y'_r = H * Y_n^T (mod 2). Если S = 0, ошибки отсутствуют. Если S != 0, синдром сопоставляется со столбцами матрицы H: совпадение с j-м столбцом указывает на позицию одиночной ошибки, исправляемой инверсией соответствующего бита.
Для достижения d_min = 4 матрица дополняется строкой общей четности. В этом случае нечетный вес синдрома указывает на одиночную ошибку (исправляется), а четный ненулевой вес — на двукратную ошибку (обнаруживается без риска ложного исправления).

2. Многомерные итеративные коды

Итеративный код строится упорядочиванием информационных бит в k1 x k2 (2D) либо k1 x k2 x z (3D) матрицу. Независимые проверочные биты вычисляются по нескольким геометрическим направлениям:

- Горизонтальные паритеты X_h: свертка по модулю 2 каждой строки матрицы;
- Вертикальные паритеты X_v: свертка по модулю 2 каждого столбца матрицы;
- Первые диагональные паритеты X_d1: свертка элементов по главным диагоналям r + c = const;
- Вторые диагональные паритеты X_d2: свертка элементов по антидиагоналям r - c = const;
- Пространственные Z-паритеты: свертка одноименных ячеек между плоскостями z;
- Контрольная сумма X_hv: общая проверка четности всех информационных бит.

В классических 2D-кодах (только строки и столбцы) ошибки, расположенные в вершинах прямоугольника, взаимно компенсируют изменения паритетов и не обнаруживаются. Добавление диагональных и пространственных проверок устраняет эту неоднозначность, позволяя мажоритарному декодеру надежно исправлять ошибки высокой кратности.

Программная реализация

Программное обеспечение реализовано в виде автономного скрипта lab2.py на языке Python (без внешних зависимостей, только стандартные модули). Код разделен на два вычислительных ядра:

1) Модуль кода Хемминга (build_hamming_H, encode_hamming, decode_hamming) — строит каноническую матрицу H, вычисляет проверочные символы, вносит ошибки и корректирует их через вектор синдрома S.
2) Класс IterativeCodeEngine — параметризуется размерами матрицы (k1, k2, z) и числом групп паритетов (2..5), выполняет матричную визуализацию, расчет паритетов и многоитеративное мажоритарное декодирование по пересечению нарушенных проверок четности.

Построение матрицы Хемминга и декодирование по синдрому:

```
def build_hamming_H(k: int, d_min: int = 3) -> Tuple[List[List[int]], int, int]:
    r_base = calc_hamming_r(k)
    p_cols = []
    wt = 2
    while len(p_cols) < k and wt <= r_base:
        for num in range(3, 1 << r_base):
            bin_str = format(num, f'0{r_base}b')
            if bin_str.count('1') == wt:
                p_cols.append([int(ch) for ch in bin_str])
                if len(p_cols) == k: break
        wt += 1
    n_base = k + r_base
    H = [[0] * n_base for _ in range(r_base)]
    for c in range(k):
        for r in range(r_base): H[r][c] = p_cols[c][r]
    for i in range(r_base): H[i][k + i] = 1
    return H, r_base, n_base

def decode_hamming(y_n, H, k, r, d_min=3):
    y_k, y_r = y_n[:k], y_n[k:]
    y_r_prime = [sum(1 for j in range(k) if y_k[j] and H[i][j]) % 2 for i in range(r)]
    syndrome = [1 if y_r[i] != y_r_prime[i] else 0 for i in range(r)]
    # Сопоставление синдрома со столбцами H для локализации ошибки
    error_pos = next((c for c in range(k + r) if [H[row][c] for row in range(r)] ==
syndrome), None)
    e_n = [0] * (k + r)
    if error_pos is not None: e_n[error_pos] = 1
    return {'syndrome': syndrome, 'E_n': e_n, 'Y_n_corrected': [y_n[i] ^ e_n[i] for i in
range(k + r)]}
```

Мажоритарное итеративное декодирование многомерного кода:

```
def decode(self, y_n: List[int], max_iters: int = 12) -> Dict[str, object]:
    y_corr = y_n[:]
    flipped = []
    for _ in range(max_iters):
        syndrome = self.compute_syndrome(y_corr)
        if sum(syndrome) == 0: break
        # Голосование: подсчет нарушенных проверок для каждого информационного бита
        best_bit, best_score, best_ratio = -1, 0, 0.0
        for b_idx in range(self.k):
            checks = self.bit_to_checks[b_idx]
            violated = sum(syndrome[c] for c in checks)
            ratio = violated / len(checks)
            if violated >= 2 and ratio > 0.5 and (violated > best_score or (violated ==
best_score and ratio > best_ratio)):
                best_score, best_ratio, best_bit = violated, ratio, b_idx
        if best_bit != -1:
```

```
        y_corr[best_bit] ^= 1
        flipped.append(best_bit)
    else: break
return {'error_detected': any(syndrome), 'corrected_word': y_corr, 'flipped': flipped}
```

```
Часть 1 — Код Хемминга (d_min = 3)

================================================================================
ЧАСТЬ 1. КОД ХЕММИНГА (d_min = 3)
================================================================================
Исходное текстовое сообщение : 'BSUIR'
Параметры кода (n, k, r)     : k = 40, r = 6, n = 46, R = k/n = 0.8696
Информационное слово X_k     : 0100001001010011010101010100100101010010
Проверочные символы X_r      : 010110
Кодовое слово X_n            : 0100001001010011010101010100100101010010010110

Проверочная матрица Хемминга H_(n,k) = [P' | I]:
  Строка  1: 0000000000111110000000000111111111100000 | 100000
  Строка  2: 0000001111000010000111111000000111101111 | 010000
  Строка  3: 0001110001000100111000111000111000110111 | 001000
  Строка  4: 0110010010001001011011001011001001011011 | 000100
  Строка  5: 1010100100010001101101010101010010011101 | 000010
  Строка  6: 1101001000100001110110100110100100011110 | 000001
--------------------------------------------------------------------------------
[Эксперимент: кратность ошибки = 1] Истинные позиции ошибок: 18 (инд. 17)
  Принятое слово Y_n         : 0100001001010011000101010100100101010010010110
  Синдром S = Y_r XOR Y'_r   : 001101
  Унарный вектор ошибки E_n  : 0000000000000000010000000000000000000000000000
  Результат анализа синдрома : Локализована одиночная ошибка в позиции 18 (индекс 17)
  Слово после коррекции Y'_n : 0100001001010011010101010100100101010010010110
  Совпадение Y'_n == X_n     : ДА (успешно восстановлено)
```

Рисунок 1 — Протокол работы кода Хемминга (d_min = 3) при одиночной ошибке

```
Часть 1 — Расширенный код Хемминга (d_min = 4)

================================================================================
ЧАСТЬ 1. КОД ХЕММИНГА С РАСШИРЕННОЙ ЧЕТНОСТЬЮ (d_min = 4)
================================================================================
Исходное текстовое сообщение : 'BSUIR'
Параметры кода (n, k, r)     : k = 40, r = 7, n = 47, R = k/n = 0.8511
Кодовое слово X_n            : 01000010010100110101010101001001010100100101101
--------------------------------------------------------------------------------
[Эксперимент: кратность ошибки = 2] Истинные позиции ошибок: 12 (инд. 11), 35 (инд. 34)
  Принятое слово Y_n         : 01000010010000110101010101001001011100100101101
  Принятая инф. часть Y_k    : 0100001001000011010101010100100101110010
  Принятые пров. биты Y_r    : 0101101
  Вновь вычисленные Y'_r     : 0110111
  Синдром S = Y_r XOR Y'_r   : 0011010
  Результат анализа синдрома : Обнаружена двукратная ошибка (синдром S != 0, не совпадает ни с одним столбцом H)
  Слово после коррекции Y'_n : 01000010010000110101010101001001011100100101101
  Совпадение Y'_n == X_n     : НЕТ (исправление невозможно, ложная коррекция заблокирована)
```

Рисунок 2 — Обнаружение двукратной ошибки расширенным кодом Хемминга (d_min = 4)

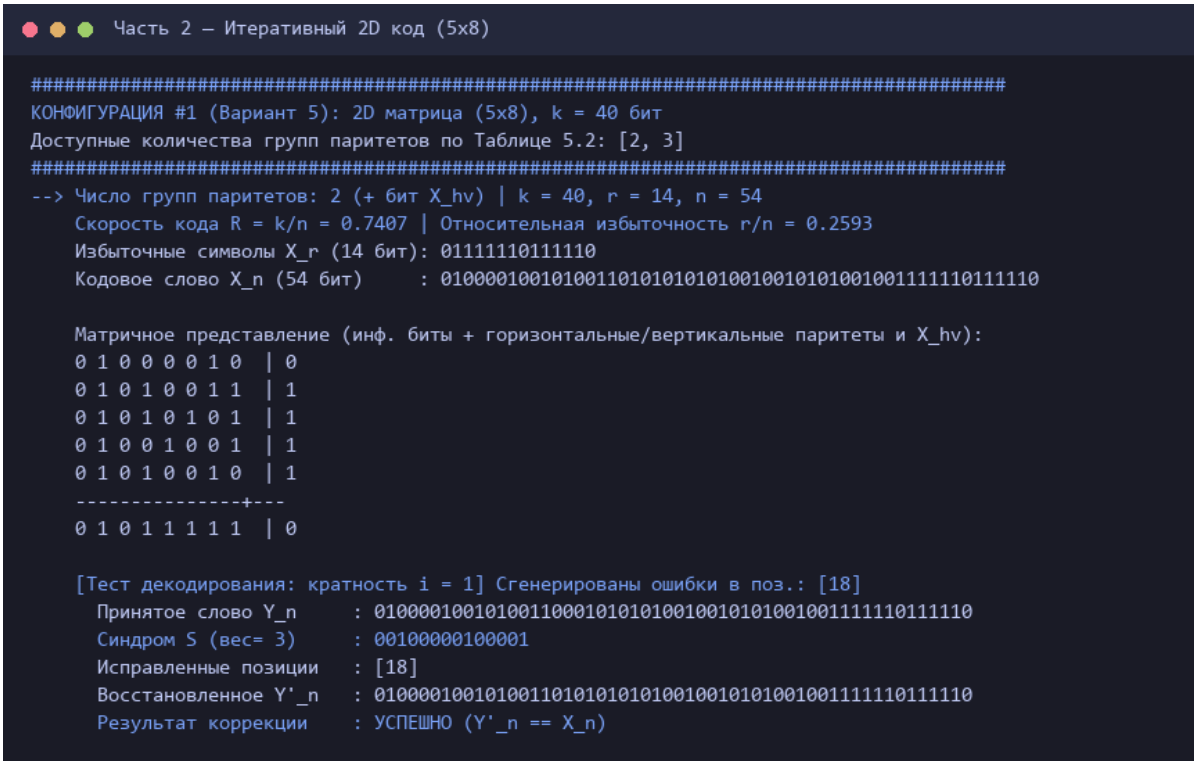


Рисунок 3 — Матричная структура 2D-кода (5x8) и исправление ошибки

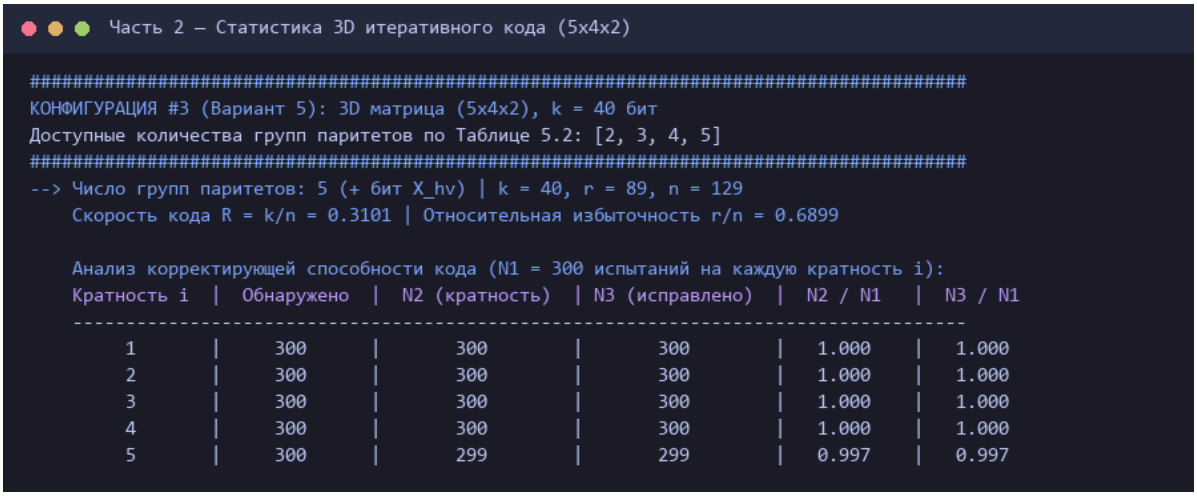


Рисунок 4 — Статистика обнаружения и коррекции ошибок для 3D-кода (5x4x2, 5 групп)